Understanding Hardware-Assisted Memory Safety

An Experimental Study of ARM Memory Tagging and Software Sanitization

Final Project Report — E0-256 Computer Systems Security

Suman Roy (SR No: 28164)
Indian Institute of Science, Bengaluru

November 2026

Abstract

This project compares five defense configurations on a common set of C programs with known memory-safety bugs: an unprotected baseline, compiler hardening, AddressSanitizer (ASan), and ARM's Memory Tagging Extension (MTE) in synchronous and asynchronous mode. It builds on Noh et al., *ARM MTE Performance in Practice* (USENIX Security 2026). It reproduces their software configuration: glibc heap tagging is enabled through a tunable on an unmodified binary. It then extends their evaluation with a detection study, four boundary cases, controlled tag-space experiments and performance profiling.

Against the six-program suite, ASan detects all six violations. MTE detects three: the heap overflow, the use-after-free and the double-free. It misses the stack overflow, because glibc tags only heap memory. It also deterministically misses both out-of-bounds accesses, because they land in the padding of a 16-byte tag granule. The boundary cases show that the two mechanisms fail on different bugs. ASan misses a non-linear overflow that jumps into a live object, and a use-after-free whose memory has left its quarantine; MTE catches both about 90% of the time. Neither catches an intra-object overflow that hijacks a function pointer. Only index bounds instrumentation stops that one.

Asynchronous MTE detects the same bugs but reports no fault address and lets the illegal store commit before reporting. Under QEMU, accidental tag matches after reallocation occur 12.3% of the time instead of the ideal 6.7%. We trace this to correlated tags from the emulated random-tag instruction (`IRG`). Natively, ASan costs 1.01–21.5× in runtime across our workloads. MTE runtime could only be measured under emulation, so we interpret those numbers against the hardware results of Noh et al.

1 Introduction

Memory-safety violations are still a leading cause of exploitable vulnerabilities in systems software. Examples are stack and heap buffer overflows, out-of-bounds (OOB) accesses, use-after-free (UAF) and double-free. Software tools such as AddressSanitizer [2] detect many such bugs, but they cost time and memory. That cost generally confines them to testing. ARM's Memory Tagging Extension (MTE) [3] offers a different design point. Each 16-byte granule of memory carries a 4-bit tag, and each pointer carries a 4-bit tag in its top byte. The hardware checks the two on every access. Noh et al. [1] recently gave the first comprehensive study of MTE's *performance* on real silicon: Pixel 8/9 cores, AmpereOne and Apple M5.

This project studies the *security* side of the same trade-off and relates it to performance. It addresses the four research questions of the proposal:

RQ1 **Detection coverage:** how does MTE compare with ASan on common spatial and temporal violations?

RQ2 **Detection boundaries:** which classes are caught by MTE only, by ASan only, or by neither?

RQ3 **Execution mode trade-offs:** how do synchronous and asynchronous MTE differ in error reporting and performance?

RQ4 Security–performance trade-off: how do runtime and memory overheads compare across allocation-heavy and access-heavy workloads?

Contributions.

1. An evaluation harness for five configurations (Section 3). It runs one vulnerability suite and records how each run ended: the signal, the `si_code` and the source location.
2. A detection matrix (Table 3) filled from 1,400 observed runs, not from assumed guarantees.
3. Four boundary cases that separate byte-precise software checking from probabilistic hardware tagging.
4. Tag-space experiments that measure detection probability against overflow distance and after reallocation. They expose a tag-generator correlation in emulation.
5. Runtime and memory profiling on allocation-heavy, access-heavy and key-value workloads.

2 Background

Compiler hardening. `-fstack-protector-strong` places a random canary between local arrays and the saved return state. The canary is checked on function return, so the program has already run past the bug when the overflow is detected. `_FORTIFY_SOURCE` replaces calls such as `strcpy` with checked variants when the compiler knows the destination size. `-fsanitize=bounds-strict` (here with `-fsanitize-trap`, so it needs no runtime) checks array *indices* against declared array bounds. These defenses are cheap but cover only narrow patterns.

AddressSanitizer. ASan maps every 8 bytes of application memory to one shadow byte. It surrounds stack, global and heap objects with poisoned *redzones*, and it instruments every load and store with a shadow check. Freed memory is poisoned and held in a FIFO *quarantine*, 256 MiB by default on x86-64, before it can be reused. ASan is deterministic and byte-precise within its model. That model has two gaps: an access that skips the redzone and lands in another valid object, and a dangling pointer used after its memory has left quarantine.

ARM MTE. MTE (Armv8.5-A) associates a 4-bit *allocation tag* with each 16-byte granule. Bits 56–59 of a pointer hold a *logical tag*, which the Top-Byte-Ignore feature makes possible. `IRG` generates a random tag, `STG/ST2G/DC GVA` write tags to memory, and `LDG` reads them. On a mismatch, three reporting modes are available:

- **SYNC** raises a precise `SIGSEGV` (`SEGV_MTESERR`) on the faulting instruction.
- **ASYNC** only sets a flag (`TFSR`), which the kernel turns into `SEGV_MTEAERR` at the next kernel entry, with no fault address.
- **ASYMM** is synchronous for loads and asynchronous for stores.

On Linux, glibc tags heap allocations when the `glibc.mem.tagging` tunable is set. Bit 0 enables tagging and bit 1 selects precise (SYNC) faults. Because MTE relies on random 4-bit tags, it is *probabilistic*: a stale or out-of-bounds pointer goes undetected if its tag happens to match.

Noh et al. [1]. The paper evaluates MTE on SPEC CPU and on server workloads (RocksDB, Nginx, PostgreSQL, Memcached). The binaries are built with Clang 18 and statically linked with glibc 2.36, and MTE is enabled through `glibc.mem.tagging`. Overheads are often modest, but every microarchitecture has outliers:

- up to 6.64× on 456.hmmer on the Pixel Performance core in SYNC mode, attributed to store serialization;

Configuration	Build flags / runtime environment	Platform
Baseline	`-fno-stack-protector -D_FORTIFY_SOURCE=0 -fno-stack-clash-protection`	both
Hardening	`-fstack-protector-strong -D_FORTIFY_SOURCE=3` `-fsanitize=bounds-strict -fsanitize-trap=all`	both
ASan	`-fsanitize=address -fno-omit-frame-pointer`, `ASAN_OPTIONS=detect_leaks=0`	x86-64
MTE-sync	baseline binary + `GLIBC_TUNABLES=glibc.mem.tagging=3`	AArch64
MTE-async	baseline binary + `GLIBC_TUNABLES=glibc.mem.tagging=1`	AArch64

Table 1: Defense configurations. The vulnerability programs are compiled at `-O0`; the benchmarks at `-O2`.

- up to 1.82× even in ASYNC mode on the Pixel Big core, attributed to a structural hazard on in-flight tag checks;
- a geometric mean of 1.10× on AmpereOne in SYNC mode, with 456.hmmer at 1.43×, attributed to store-to-load forwarding.

They also show that tagging large allocations, i.e. the cost of `STG`, is visible on some workloads. Tagging only allocations below 32 KiB reduced 403.gcc from 1.23× to 1.15× on AmpereOne. Their artifact (`https://github.com/UT-Security/mte-root`) targets MTE hardware: Android Pixel devices and AmpereOne.

3 Experimental Setup

Platforms. No MTE-capable hardware was available for this project. The proposal planned for this case: QEMU serves as the functional-validation environment for MTE and is not treated as a substitute for hardware timing. Two platforms were used:

- **x86-64 native:** AMD Ryzen 5 5600H, Linux, GCC 15.2, glibc 2.43. It runs Baseline, Hardening and ASan for detection and for real performance numbers.
- **AArch64 with MTE:** cross-compiled with `aarch64-linux-gnu-gcc` 15.2 and run under QEMU 10.2 user-mode emulation with `-cpu max`, which implements MTE. It runs Baseline, Hardening, MTE-sync and MTE-async.

ASan could not run under QEMU user mode, because its shadow-memory reservation fails, so ASan results come from x86-64. ASan's checking logic does not depend on the architecture.

Each platform has its own Baseline and Hardening runs, which serve as controls. The MTE configurations run the *same* AArch64 baseline binary with tagging switched on, so any difference from the AArch64 baseline is caused by MTE alone, not by the architecture or by QEMU. For the same reason all AArch64 slowdowns are relative to the AArch64 baseline under QEMU, and ASan's are relative to the x86-64 baseline. Neither platform can replace the other here: ASan, real timings and `perf` counters exist only on x86-64, and MTE exists only on AArch64.

glibc version. The Ubuntu cross sysroot ships glibc 2.43. With heap tagging enabled, its `free()` raised a tag-check fault on *every* call, even in a correct `malloc`/`free` program. The fault was a synchronous fault on glibc's own validation load in `__libc_free`. Every MTE result would therefore have been a false positive. All AArch64 runs instead use Debian 12's glibc 2.36, which is the version Noh et al. used. It is unpacked into a private sysroot and passed to QEMU with `-L`. Binaries are compiled with `-std=gnu17` so that they do not need C23 symbol versions newer than 2.36.

Configurations. Table 1 lists the five configurations. Ubuntu's GCC enables stack protection and fortification by default, so the baseline disables them explicitly. As in Noh et al., MTE needs *no recompilation*: the two MTE configurations execute the baseline binary.

Program	Violation
`stack_overflow`	`strcpy` of 21 bytes into `char buffer[10]` on the stack
`heap_overflow`	`strcpy` of 21 bytes into `malloc(10)`
`oob_read`	read `arr[5]` of `malloc(5*sizeof(int))` (byte offset 20 of a 20-byte object)
`oob_write`	write `arr[5]` of the same allocation
`use_after_free`	`free(array); array[0]=42;`
`double_free`	`free(ptr); free(ptr);`
B1 `intra_granule_overflow`	write byte 12 of `malloc(10)`: out of bounds, but inside the object's 16-byte granule
B2 `far_oob_into_live_object`	`a[idx]` with `idx` chosen to land inside a second live object `b`, skipping ASan's redzone
B3 `uaf_after_reallocation`	free a chunk, churn 320 MiB to flush ASan's quarantine, re-allocate 20,000 objects (one reuses the chunk), then write through the stale pointer
B4 `intra_object_overflow`	overflow `name[16]` inside a heap `struct` into its adjacent function pointer, then call it (control-flow hijack)

Table 2: Vulnerability suite (top) and boundary cases (bottom).

Optimisation level. At `-O1` and above, GCC deletes most of the suite's bugs. For example, `malloc/free/array[0]=42` in `use_after_free.c` folds to nothing, and the never-read `strcpy` into a dead stack buffer is removed. Undefined behaviour lets the compiler assume the bug never happens. The suite is therefore compiled at `-O0` for *every* configuration, so each defense sees the same faulty accesses. A consequence is that `_FORTIFY_SOURCE` is inactive in the detection runs, because it needs optimisation. In a separate check at `-O2 -fno-builtin`, it caught the heap-buffer `strcpy` (`*** buffer overflow detected ***`).

Harness. A small `fault_reporter.c` is linked into every program, so the suite sources stay unchanged. It installs `SA_SIGINFO` handlers and prints the signal, the `si_code` (which separates MTE SYNC from ASYNC), the fault address and the PC offset. A Python runner maps the PC to `file:line` with `addr2line`, or to the nearest symbol. It classifies every run as *detected* (with the mechanism), *crashed* (a segfault that is a side effect, not a report), or *missed* (the program completed). The classification uses the ASan report, the glibc abort messages and the fault line. Every (program, configuration) pair was run 20 times, because MTE tags and allocator reuse are random. Runtime is the median wall-clock time of 5 runs after a warm-up run, and memory is `ru_maxrss` from `wait4`. On x86-64, Linux `perf stat` additionally records cycles, instructions, cache references and misses, branch misses and page faults (mean of 5 runs after a warm-up). `perf` is not used for the QEMU runs, because there it would count the emulator's own instructions rather than the guest program's.

4 Phase 1: Vulnerability Suite and Boundary Cases

Table 2 lists the programs. The six suite programs are those from the proposal, and each has a known bug type and location. The four boundary cases were added for RQ2 and Phase 4. Each is built so that the defenses *should* disagree on it.

5 Results

5.1 RQ1: Detection coverage

Table 3 and Figure 1 summarise all 1,400 runs.

Vulnerability	Baseline		Hardening		ASan	MTE-sync / MTE-async	Analysis
	x86	A64	x86	A64	x86	A64	
Stack buffer overflow	!crash	✗silent	✓canary	✓canary	✓ASan	✗silent / silent	Only stack defenses (canary, ASan) see it; glibc MTE tags heap only
Heap buffer overflow	✗silent	✗silent	✗silent	✗silent	✓ASan	✓tag fault / async fault	Spatial: crosses a 16-B granule ⇒ MTE detects
Out-of-bounds read	✗silent	✗silent	✗silent	✗silent	✓ASan	✗silent / silent	Spatial: lands in granule padding ⇒ MTE misses
Out-of-bounds write	✗silent	✗silent	✗silent	✗silent	✓ASan	✗silent / silent	Spatial: lands in granule padding ⇒ MTE misses
Use-after-free	✗silent	✗silent	✗silent	✗silent	✓ASan	✓tag fault / async fault	Temporal: freed memory retagged ⇒ MTE detects
Double-free	✓glibc check	✓glibc check	✓glibc check	✓glibc check	✓ASan	✓tag fault / async fault	Allocator interaction: glibc tcache check and MTE both catch
B1 Intra-granule overflow	✗silent	✗silent	✗silent	✗silent	✓ASan	✗silent / silent	Byte-precise ASan vs 16-B MTE granularity
B2 Far OOB into live object	✗silent	✗silent	✗silent	✗silent	✗silent	✓tag fault (18/20) / async fault	Jumps ASan redzone; MTE tag mismatch
B3 UAF after reallocation	✗silent	✗silent	✗silent	✗silent	✗silent	✓tag fault (19/20) / async fault	ASan quarantine evicted; MTE probabilistic
B4 Intra-object overflow	✗hijacked	✗hijacked	✓bounds trap	✓bounds trap	✗hijacked	✗hijacked / hijacked	Inside one object: only index bounds check; CFI territory

Table 3: Comparative evaluation matrix (proposal Table 1), filled from 20 runs per cell. ✓ = detected (mechanism), ✗ = missed (the program ran to completion), ! = crashed without a report. A fraction gives the detected runs out of 20 when not all agreed. For MTE, the left entry is SYNC and the right entry is ASYNC.

Baseline. Without protection, five of the six suite bugs execute silently and the program prints "Done.". The exception is the double-free. glibc's tcache already detects it (`free(): double free detected in tcache 2`). This is an allocator integrity check, not a general memory-safety mechanism. On x86-64 the stack overflow later crashes on `ret` (`SEGV_MAPERR`), because the saved return address is overwritten. On AArch64 it runs to completion. There the link register is saved *below* the locals, so the overflow corrupts the caller's frame instead and the program survives. The outcome depends on the architecture's frame layout.

Compiler hardening. The stack canary detects the stack overflow on both architectures (20/20). It does so only at function return, *after* the overflowing `strcpy` and the following `printf` have executed: "Done." is printed in every run. Hardening offers nothing for the heap bugs at `-O0`.

ASan detects all six suite violations (20/20 each) and reports the exact source line. It also names the right bug class: `stack-buffer-overflow`, `heap-buffer-overflow`, `heap-use-after-free` and `attempting double-free`.

MTE (both modes) detects three of the six suite bugs, each 20/20.

- **The heap overflow**: the 21-byte copy crosses from the 16-byte granule of `malloc(10)` into the next chunk's header granule.
- **The use-after-free**: glibc re-tags freed memory to tag 0, so the stale pointer mismatches.
- **The double-free**: the second `free` performs a tag-checked load inside glibc's `free`.

It misses three:

- **The stack overflow**: glibc's MTE support tags only the heap. Stack tagging needs compiler support (Clang's `-fsanitize=memtag-stack`), which GCC 15 does not provide.
- **The out-of-bounds read and write**: `malloc(20)` is rounded up to 32 bytes, i.e. two granules with the same tag. The access at byte offset 20 lands in the object's own padding, so the tags match. This miss is *deterministic*, not probabilistic.

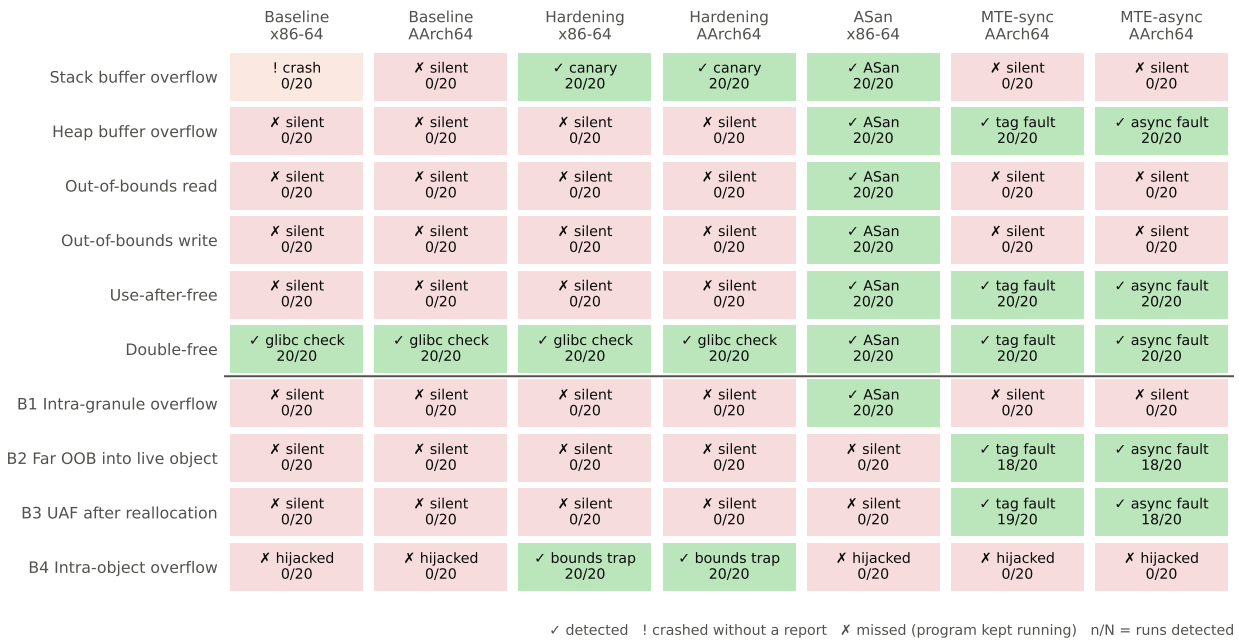


Figure 1: Observed outcome of every program in every configuration (20 runs per cell).

A bug that is out of bounds for the language can still be in bounds for the tag. **MTE's spatial resolution is 16 bytes, not 1 byte.**

5.2 RQ2: Detection boundaries

The four boundary cases separate the mechanisms cleanly (Table 3, bottom).

B1 (intra-granule): ASan detects it (20/20), because its shadow encoding records partially addressable 8-byte words. MTE misses it (0/20) for the granule reason above. Figure 2a measures this boundary directly. For every request size from 1 to 64 bytes, the first faulting byte offset under MTE is exactly $\lceil n/16 \rceil \cdot 16$, in 100% of the probes. This leaves up to 15 bytes of undetectable overflow. ASan detects at offset n.

B2 (far OOB into a live object): ASan misses it (0/20). The target address belongs to a valid object, so its shadow is clean. MTE detects it in 18/20 runs in each mode, because `a`'s pointer tag differs from `b`'s memory tag except when they collide by chance.

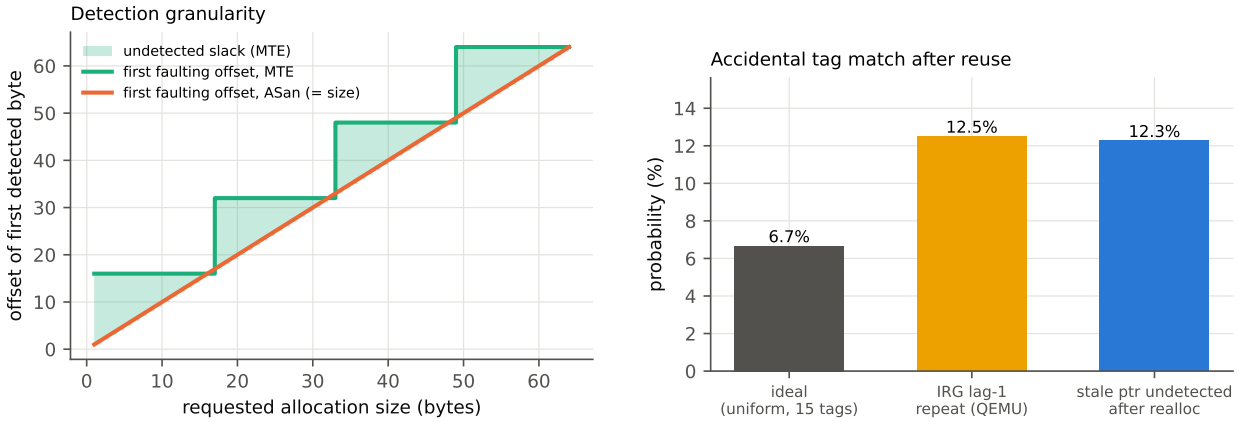
B3 (UAF after reallocation): After 320 MiB of churn, the freed chunk was reused by a live object in all 20 runs, and ASan reported nothing (0/20). Its quarantine had been exhausted. MTE re-tags on every allocation and detects it in 19/20 runs (SYNC) and 18/20 (ASYNC). The misses are accidental tag matches (Section 5.5).

B4 (intra-object overflow): No baseline, ASan or MTE run detects it. In every run the overwritten function pointer is called, printing `CONTROL FLOW HIJACKED`. ASan places no redzones inside a `struct`, and MTE tags the whole allocation with one tag. Only `-fsanitize=bounds-strict` traps it (20/20), because it knows `name[]` has 16 elements.

B4 shows that memory tagging is *not* a control-flow defense (course Topic 5). An attacker who can corrupt a code pointer *within* its own object keeps full control. That gap is what CFI and pointer authentication are for.

5.3 RQ3: Synchronous vs. asynchronous MTE

Detection. On the suite, the two modes detect exactly the same bugs. The differences are in *what* is reported and *when*. In ASYNC mode the fault always carries `si_addr = 0`. The PC we



(a) Detection granularity (MTE-sync vs. ASan). (b) Accidental tag match after reuse (40,000 trials).

Figure 2: Phase 4: MTE granularity and tag reuse.

Access	Mode	`si_code`	fault address	illegal store committed	leaked bytes used
store	SYNC	9 `SEGV_MTESERR`	exact (tagged)	no	–
store	ASYNC	8 `SEGV_MTEAERR`	`0`	**yes**	–
load	SYNC	9 `SEGV_MTESERR`	exact (tagged)	–	0
load	ASYNC	8 `SEGV_MTEAERR`	`0`	–	**1**

Table 4: Synchronous vs. asynchronous reporting (QEMU, 10/10 runs identical).

recorded is the place where the fault was delivered, not where the bug is. In the suite it was the next library call after the bug (`puts@plt`, `printf@plt`, or glibc's `free`). SYNC mode instead points at the source line of the bad access, e.g. `use_after_free.c:12`.

Mode probe. `mte_mode_probe` performs a single illegal store, or 16 illegal loads, into a neighbouring object that holds a secret. It then runs a 50-million-iteration loop with no system calls. Table 4 shows the outcome, which was identical in all 10 repetitions. In SYNC mode the bad instruction never retires: the secret is intact and no leaked byte reaches the program. In ASYNC mode the illegal store *has already modified the victim object* by the time the fault arrives, and the first leaked byte was already consumed. QEMU delivers the asynchronous fault at the next translation-block boundary, after at most one loop iteration. On hardware, Linux delivers it only at the next kernel entry: a system call, an interrupt or a context switch. The window between corruption and report is therefore *longer* on real hardware than what we observed. ASYNC therefore detects corruption after the fact rather than preventing it. It makes exploits unreliable and detectable, but a single committed write may already be enough for an attacker, and the report gives no address for diagnosis.

Performance. Under emulation, MTE-async runs at 1.01× the time of MTE-sync (geometric mean over workloads). The two are close because QEMU checks tags in software in both modes. This does *not* transfer to hardware. On hardware the modes differ in how the pipeline orders the tag check against the access. Noh et al. found SYNC up to 6.64× slower on one core, and ASYNC still up to 1.82× on another, from microarchitectural causes that emulation cannot model.

5.4 RQ4: Performance and memory overhead

Native x86-64 (Figure 3, Table 5). *Compiler hardening* stays between 0.90× and 1.09× of the baseline. `perf` shows that these deviations are run-to-run noise: hardening executes 1.00–1.02× the baseline's instructions. The benchmarks access memory through pointers, which `bounds-strict` cannot check, so almost no checks are added.

ASan's cost depends strongly on the workload:

- **Allocation-heavy:** 16.2× (small objects), 12.3× (1–4 KiB) and 21.5× (32–128 KiB). Every `malloc/free` poisons and unpoisons shadow memory, adds redzones and pushes memory through the quarantine. Peak memory grows by up to 24.6×, because the 256 MiB quarantine plus redzones dwarf the small live heap.
- **Key-value store (mixed):** 2.07× runtime and 11.8× peak memory.
- **Memory-bound kernels:** ASan costs only 1.03× on the streaming loop and 1.01× on the pointer chase. Its per-access shadow check is hidden behind DRAM bandwidth and latency.

ASan's overhead is therefore dominated by *allocator* work and *memory* footprint, not by access checking.

Hardware counters (`perf stat`, Table 6). The counters explain where ASan's time goes:

- **Small allocations:** ASan executes 20× the instructions and causes 1,266× the page faults. glibc keeps reusing a few hot chunks, while ASan's quarantine keeps handing out fresh memory, so new pages (and their shadow) are touched all the time.
- **Large allocations:** the slowdown is larger than the instruction increase, and IPC falls from 0.83 to 0.30. Cache misses rise 78×: poisoning the shadow and redzones of 32–128 KiB objects and cycling them through the quarantine is memory traffic, not computation.
- **Memory-bound kernels:** ASan executes 1.77× (streaming) and 1.91× (pointer chase) the instructions, yet cycles change only 1.00× and 1.02×. The extra shadow-check instructions run in the shadow of DRAM stalls (the pointer chase has an IPC of only 0.03), so they are nearly free.
- **Key-value store:** 3.4× instructions and 16× page faults, consistent with a mix of the two cases above.

This is the software analogue of the effect Noh et al. report for MTE: the cost of a memory-safety mechanism comes mostly from *allocation-time* work and from memory-system effects, rather than from the per-access check itself.

AArch64 under QEMU (Figure 4). These numbers are emulation costs: QEMU implements every tag check and every `STG` in software. They must not be read as MTE hardware overhead; Noh et al. report 1.0–1.43× on AmpereOne and up to 6.64× on one Pixel core. The relative pattern is still informative, because it shows *which* operations MTE adds work to. Cost rises with the number of bytes tagged per allocation:

- 8.1× for small objects;
- 14.4× for 1–4 KiB objects;
- 83× for 32–128 KiB objects, where glibc must tag the whole object on `malloc` and re-tag it on `free`.

This is qualitatively the effect Noh et al. isolate in their §4.4.4, where tagging large allocations is expensive and skipping tags above 32 KiB reduces overhead. Every access is also tag-checked, so the streaming loop costs 14.9× in emulation, while the latency-bound pointer chase costs only 1.9×.

Peak RSS grows by 2.3–4.0% with MTE in every workload, which matches the architectural tag-storage ratio of 4 bits per 16 bytes = 3.125%. QEMU allocates that storage for tagged pages; on hardware it is carved out of DRAM and does not show in RSS. MTE does not inflate the heap the way ASan's redzones and quarantine do: 1.0× against up to 24.6×. SYNC and ASYNC are indistinguishable under emulation (Section 5.3).

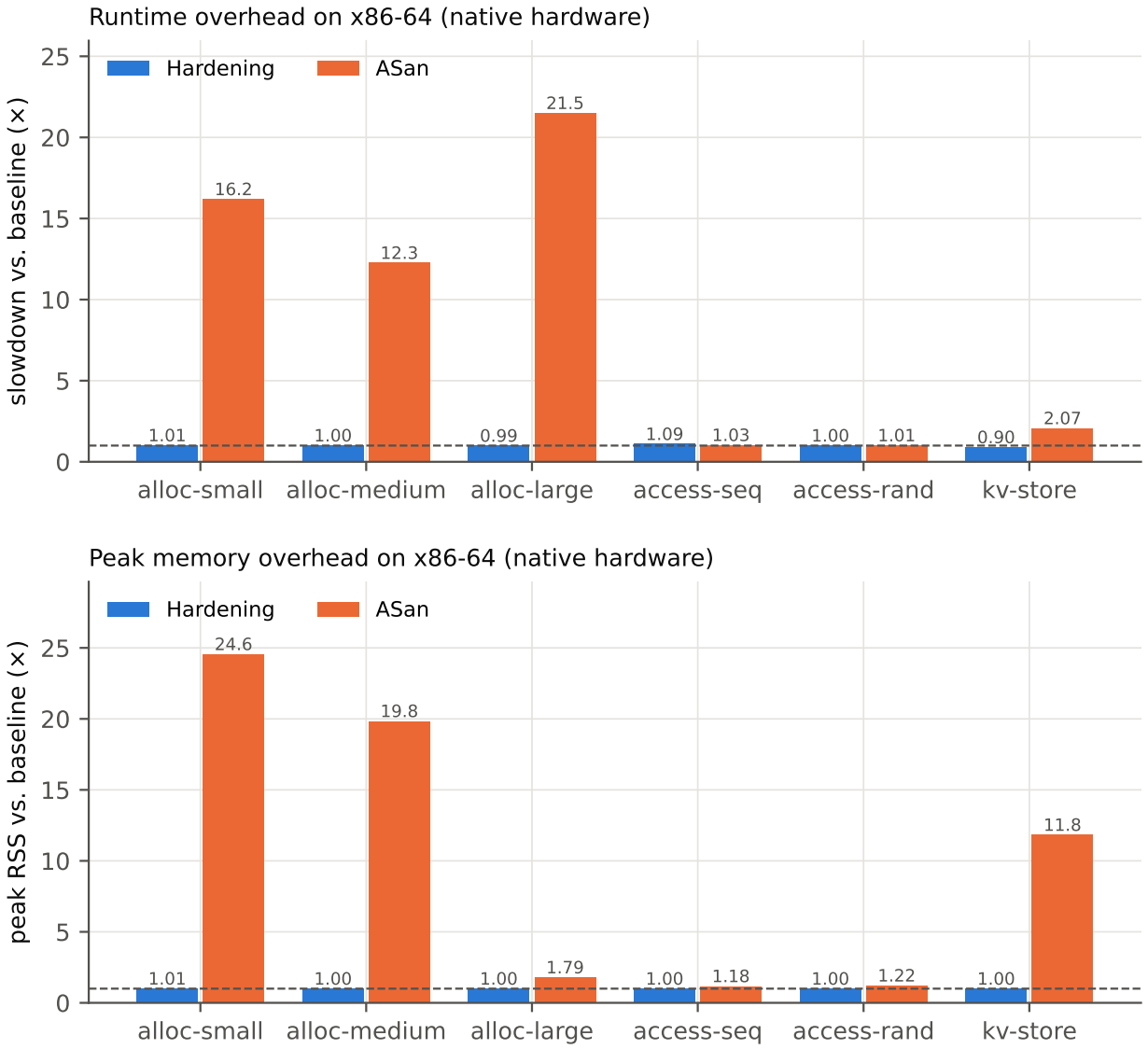


Figure 3: Native x86-64: runtime and peak memory relative to the baseline (median of 5 runs).

5.5 Phase 4: The finite tag space

`mte_tag_space` runs in SYNC mode. It catches each tag-check fault with `sigsetjmp`/`siglongjmp`, so tens of thousands of single-load probes run in one process. Loads are used so that undetected probes never corrupt the heap.

Tag distribution. Across 200,000 allocations, tag 0 was never handed out, and tags 1–15 occurred between 12903 and 13565 times each, i.e. roughly uniformly. glibc reserves tag 0 for chunk headers and freed memory.

Detection vs. distance. Figure 5 probes an OOB load d bytes past the end of a 48-byte object surrounded by live neighbours. Each point is 2,000 trials, and every object is re-allocated between trials so that all tags are re-randomised. Three regimes appear:

- Every 64 bytes the probe lands on a chunk header (tag 0), and detection is **100%**.
- Elsewhere in far-away objects it averages **93.4%**, close to the ideal $14/15 = 93.3\%$.
- Overflows into the *immediately adjacent* object are detected only **85.8–89.1%** of the time.

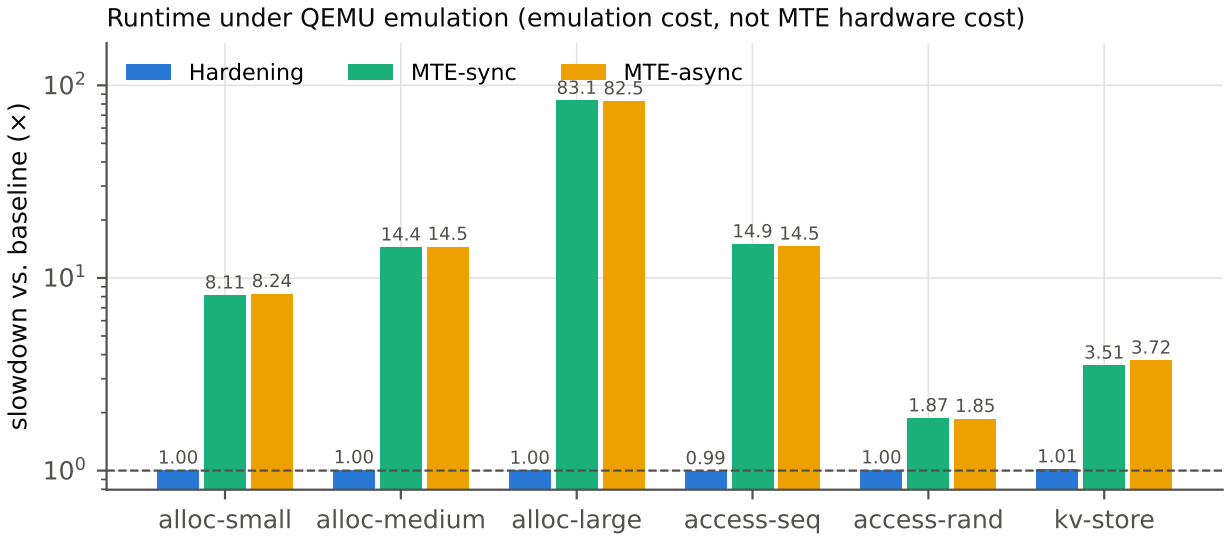


Figure 4: AArch64 under QEMU (log scale). These numbers measure the *emulation* cost of software tag checking and tag setting. They are not MTE hardware overheads, but they show which operations MTE adds work to.

	x86-64 native: slowdown / RSS			AArch64 under QEMU: slowdown (RSS)					
Workload	base (s)	Hard.	ASan	base (s)	Hard.	MTE-sync		MTE-async	
alloc-small	0.51	1.01×	16.19× / 24.56×	0.46	1.00×	8.1×	(1.00)	8.2×	(1.00)
alloc-medium	0.74	1.00×	12.26× / 19.85×	0.47	1.00×	14.4×	(1.00)	14.5×	(1.00)
alloc-large	0.24	0.99×	21.50× / 1.79×	0.09	1.00×	83.1×	(1.04)	82.5×	(1.04)
access-seq	0.50	1.09×	1.03× / 1.18×	0.16	0.99×	14.9×	(1.03)	14.5×	(1.03)
access-rand	1.77	1.00×	1.01× / 1.22×	0.50	1.00×	1.9×	(1.02)	1.9×	(1.02)
kv-store	1.43	0.90×	2.07× / 11.83×	1.53	1.01×	3.5×	(1.04)	3.7×	(1.04)

Table 5: Median runtime overheads (5 runs). x86-64 shows "slowdown / peak-RSS ratio" for ASan. AArch64 RSS ratios are in parentheses and include the QEMU process.

This is the structure the paper summarises as "a 15/16 chance it will trap". Exact numbers depend on the allocator's layout and on tag generation.

Reuse and the tag generator. Freed memory is re-tagged to 0, so every stale access *before* reuse was detected (40,000/40,000). After the chunk is re-allocated, a stale access goes undetected whenever the new tag equals the old one. This happened in **12.3%** of 40,000 trials, almost twice the ideal $1/15 = 6.7\%$ (Figure 2b). A direct test of `IRG` explains it. Under QEMU, two consecutive `IRG` results are equal with probability **12.5%**, against 6.7% for independent tags, while lag-2 results are nearly independent (7.0%). A re-allocation and the previous allocation of the same chunk are consecutive `IRG` calls, and so are two adjacent objects allocated back to back. This one correlation therefore explains both the reuse rate and the weaker adjacent-object detection above. *MTE's probabilistic guarantee is only as strong as its tag generator.* This property must be validated on the actual hardware, and allocators should exclude the previous tag explicitly, as some hardened allocators do [5].

6 Discussion

Security–performance trade-off. The mechanisms are complementary, not ranked. ASan is deterministic and byte-precise near objects, covers the stack, and identifies the bug class. It cannot see wild accesses that land in valid memory, or use-after-free once memory is reused, and

	Hard.	ASan relative to baseline				
Workload	instr.	instr.	cycles	IPC (base→ASan)	cache-miss	page-faults
alloc-small	1.00×	20.2×	16.0×	1.59→2.02	1,003×	1,266×
alloc-medium	1.00×	5.0×	12.7×	1.72→0.68	61×	158×
alloc-large	1.00×	7.8×	21.6×	0.83→0.30	78×	8×
access-seq	1.00×	1.8×	1.0×	1.04→1.83	1×	1×
access-rand	1.00×	1.9×	1.0×	0.03→0.05	1×	1×
kv-store	1.02×	3.4×	2.3×	0.83→1.24	2×	16×

Table 6: `perf stat` counters on x86-64 (mean of 5 runs), relative to the baseline. "Hard. instr." is the instruction count of compiler hardening; the remaining columns are ASan.

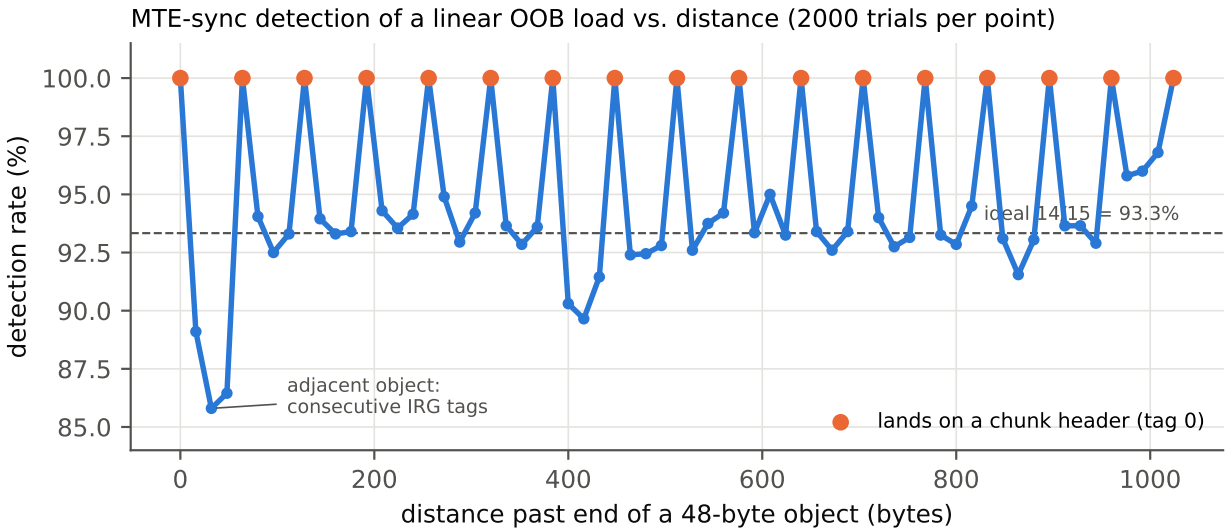


Figure 5: Detection probability of an out-of-bounds load as a function of distance (MTE-sync, 2,000 trials per point).

it costs 1.01–21.5× runtime and 1.18–24.6× memory here. That keeps it in testing. MTE needs no recompilation, and its storage cost is fixed by the architecture: 4 bits per 16 bytes, 3.125%. It also catches exactly the long-range and temporal cases that ASan misses. The price is 16-byte granularity, heap-only coverage with glibc and GCC, and probabilistic detection (roughly 88–93% per access in our measurements). In ASYNC mode, detection also comes after the corruption. Noh et al. show its hardware runtime cost is usually small but has sharp, microarchitecture-specific cliffs. Compiler hardening is nearly free but narrow. Its canary detects stack smashing only at return, and `bounds-strict` was the only mechanism that stopped the intra-object overflow (B4).

Mapping to the course.

- **Topics 1–2:** the suite demonstrates stack smashing.
- **Topic 3:** the evaluation covers software and hardware defenses.
- **Topic 4:** redzones and shadow memory are compared with 16-byte tagging.
- **Topic 5:** B4 shows a control-flow hijack that no memory-safety mechanism in our set except index checking prevents.
- **Topic 16:** the study exercises MTE instructions, modes and kernel and allocator integration.

Limitations.

1. No MTE hardware was available. MTE detection behaviour is architecturally defined and was validated in QEMU, but MTE *timing* could not be measured. The `IRG` correlation may be specific to QEMU.
2. ASan ran on x86-64 rather than AArch64.
3. GCC has no MTE stack tagging, so stack MTE (Clang `memtag-stack`) is not covered.
4. The suite programs are single-bug micro-programs compiled at `-O0`, and real code at `-O2` may behave differently.
5. ASYMM mode was not evaluated.

7 Conclusion

On the same programs, AddressSanitizer and ARM MTE catch *different* bugs. ASan found all six suite violations and the intra-granule overflow, deterministically and with exact diagnostics. MTE found the heap overflow, the use-after-free and the double-free with no recompilation. It also found, with roughly 90% probability, the far-out-of-bounds and reused-memory cases that ASan cannot see. It missed the stack and every overflow that stays within a 16-byte granule. Asynchronous MTE detects the same bugs but only after the corrupting access has taken effect, and without an address. Neither mechanism stops a control-flow hijack through an intra-object overflow. A deployment that wants low overhead in production should therefore pair MTE with stack protection and control-flow integrity, and should keep ASan for testing.

References

[1] T. Noh, Y. Wang, T. Garfinkel, M. Madhav, D. Moghimi, M. Erez, S. Narayan. *ARM MTE Performance in Practice*. 35th USENIX Security Symposium, pp. 497–516, 2026.

[2] K. Serebryany, D. Bruening, A. Potapenko, D. Vyukov. *AddressSanitizer: A Fast Address Sanity Checker*. USENIX ATC, 2012.

[3] Arm Ltd. *Armv8.5-A Memory Tagging Extension*, white paper, 2019; Arm Architecture Reference Manual for A-profile.

[4] The GNU C Library Manual, *Memory Related Tunables* (`glibc.mem.tagging`).

[5] LLVM Project. *Scudo Hardened Allocator*. `https://llvm.org/docs/ScudoHardenedAllocator.html`.